Activity 5 Report: Database Maintenance and Monitoring

COMP 018 Database Administration | Project tested: HabalGo (Ride and Booking Management System) (PHP + PostgreSQL) | Date of testing: 7 November 2026

1. Cover and links

Item	Value
Testing group	G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph
Project tested	HabalGo (Ride and Booking Management System)
Owning group	Group 6 (member names are not listed in the repository)
Repository	https://github.com/jeanaroyo/HabalGo
Tracker	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests
Evidence files	a5_health.txt, a5_logs.txt, a5_maintenance.txt

This is a re-test. The database was dropped and rebuilt from the repository files, and every result below comes from this run.

Run at about 00:24 on 7 November 2026 (Asia/Manila) after the application and access tests.

2. Health and activity

Check	Result
Version	PostgreSQL 18.6, x86_64 Windows
Database size	8654 kB
Connections	1 backend (the monitoring session). Nothing stays connected between requests. max_connections is 100
Transactions	521 commits, 57 rollbacks (from the constraint and permission tests)
Cache	68,916 buffer hits against 17 reads, about 99.98% hit ratio
Deadlocks and temp files	0 and 0
Autovacuum	on, but no table had been vacuumed or analysed yet
Rows	booking 6, customer 11, driver 10, driver_vehicle 0, location 10, payment 2, rating_feedback 1, vehicle 1
Table size	8 to 72 kB per table, mostly index pages

Dead tuples before maintenance: booking 7, customer 5, payment 5, location 3, rating_feedback 3, vehicle 3, driver 1. They come from the updates and deleted test rows and were below the autovacuum threshold, so autovacuum had not acted.

Index use during the test traffic: idx_booking_date, idx_booking_driver, idx_rating_booking, uq_rating_booking_customer, driver_license_no_key and vehicle_plate_no_key had zero scans. That is expected on a tiny data set and is not a reason to drop them.

3. Logging

Setting	Value	Comment
log_statement	none	No statement logging
log_min_duration_statement	-1	Slow queries are not logged
Server log folder	C:/Program Files/PostgreSQL/18/data/log	Readable
Application log	Console only	The PHP server writes request lines to its console and the project has no log file of its own

4. Log findings

The PostgreSQL log is shared by everything on this server, so older files also hold entries from other projects tested earlier. Only entries from 00:22 on 7 November belong to HabalGo.

Time	Entry	Explanation
00:22:14 to 00:22:18	value too long for varchar(100); null value in column "name"; duplicate key on customer_email_key and vehicle_plate_no_key; foreign key violations on booking and payment; check constraint violations on status, amount and rating; numeric field overflow; RESTRICT violations on customer and booking	The Activity 3 tests. Each one matches a call in a3_app.txt
00:22:14 and 00:22:15	syntax error at or near "WHERE" (five times)	The update calls that sent only part of a record (an empty customer body and four vehicle driver_id calls). The API builds an UPDATE with an empty SET list. It repeats at 00:23:37 for the Activity 4 vehicle call
00:22:18	syntax error at or near ")"	The malformed JSON test, which becomes an INSERT with no columns
00:23:33	role ... already exists (six times)	The role script, two runs (see Activity 4)
00:23:34 to 00:23:41	permission denied for table, schema, sequence, database; must be owner; permission denied to create role and for pg_authid	The access tests of Activity 4, all expected
00:24:35	permission denied to vacuum and analyze "booking", skipping it	habalgo_app_user attempting maintenance (section 5)

Every entry has an explanation in the test steps, and nothing was logged that was not caused by a test. Two kinds of entry are worth noting: the syntax errors show that partial updates reach the database as invalid SQL, and the log is the only place the real cause appears, because the API replies with "Database error." for all of them. The PHP server own log file stayed at 420 bytes with no warnings or errors.

5. Maintenance performed

Action	Result
ANALYZE (whole database)	Completed, last_analyze set on all 8 tables
VACUUM (VERBOSE, ANALYZE) booking	7 dead row versions removed, 6 remain, 1 page, 4 indexes cleaned
VACUUM (VERBOSE, ANALYZE) customer	5 dead row versions removed, 11 remain, 2 indexes cleaned

Dead tuples afterwards	booking 0, customer 0 (the other tables left for autovacuum)
VACUUM and ANALYZE as habalgo_app_user	Skipped, permission denied (not the table owner). The project defines no maintenance role

6. Query plans

The four queries below are the ones api.php runs for the bookings list, dashboard figures, payments list and bookings by status. They were run with EXPLAIN (ANALYZE, BUFFERS).

Query	Plan	Planning / execution
Bookings list (4 joins, newest first)	Sequential scans with hash joins, then a sort	6.53 ms / 0.23 ms
Dashboard statistics (4 subqueries)	Four sequential scans with aggregates	6.36 ms / 0.11 ms
Payments list	Hash joins over sequential scans, sort	5.02 ms / 0.12 ms
Bookings by status	Sequential scan and hash aggregate	2.36 ms / 0.09 ms

With 6 bookings a sequential scan is the right plan. To see what happens as the table grows, sequential scans were switched off for one transaction and then rolled back. The bookings list then used booking_pkey (backward, for the sort order), customer_pkey, location_pkey and driver_pkey, so the primary keys are usable once the tables are large enough for the planner to prefer them.

Foreign key columns without an index: driver_vehicle.vehicle_id, booking.pickup_location_id, booking.dropoff_location_id and rating_feedback.customer_id. Deleting a location or customer, or the vehicle lookup that api.php does by vehicle_id, would have to scan these tables once they grow.

7. Recommendations

- Create a maintenance role (member of pg_maintain) for scheduled VACUUM and ANALYZE, because the application account cannot run them.
- Set log_min_duration_statement to about 500 ms and log_statement to ddl.
- Add indexes on the four unindexed foreign key columns before the tables grow.
- Have the API return the real cause for bad input (partial updates, bad JSON), so that these syntax errors do not have to be found in the server log.
- Add a log file for the PHP side (error_log in php.ini or a small logger in api.php).
- Re-run EXPLAIN on the bookings list when there are thousands of bookings.

8. Feedback for the owners

No health problems found. The useful items are the unindexed foreign keys, the maintenance role and a clearer error for partial updates.

AI usage declaration

- Tool: Claude (Anthropic).
- Tasks: interpreting the results of the testing.
- Validation: the commands were run by the group on its own computer, and every result in this report comes from the saved output files.